

Classes and Objects:

Java achieves oops principles with the help of classes and objects.

Class: A class is a blueprint from which the individual objects are created. It represents the state and behaviour of an entity (anything that has existence with some properties and behaviour like car, student, laptop, bag, university, player, etc). A class is a logical entity

Object: An Object is the instance for a class (instance - initialization). An Object is a physical entity because it has memory allocated for all variables in the class. we can create any no of objects for the single class.

example:

let say I want to store 60 student's information like roll_num, name, age, marks. before knowing their property values, we will define a prototype suitable for them called as 'class'.

A class is declared using the keyword 'class'. all the properties and functions related to an entity are defined with in that class.

create class Student - class

```
Student{  
  
    String    roll_num;  
  
    String name;  
  
    int age;  
  
    int marks;  
  
    void writeExams() {  
  
    }  
  
    void attendClasses() {  
  
    }  
  
}    // end of class
```

Now whenever we want to store a student information, we request memory for storing the property values, the allocated memory to store student information is called as 'object/instance'.

General Form of a Class:

A class generally contain three sections - variables, constructors and methods.

Variables represent its state/properties in form of fields. Class can have static and instance variables. functionality/behaviour will be implemented in methods under a class.

Object Oriented Programming Through Java (Unit- II)

Class can have static and instance methods. Constructors will initialize the instance variables of a class when an object is created.

General form or Syntax of a Class

```
class NameOfClass {  
  
    // instance variable declaration  
  
    type1 varName1 = value1; type2  
    varName2 = value2;  
  
  
    typeN varNameN = valueN;  
  
    // Constructors  
  
    // no argument constructor  
    NameOfClass() {  
        // body of constructor  
    }  
  
    // constructor with arguments, we can define any more than one constructor to same same  
  
    NameOfClass (cparamN) {  
        // body of constructor  
    }  
    // Methods  
    static returnType1 methodName1 (mParams1){  
        // body of static method  
    }  
  
    returnTypeN methodNameN(mParamsN) {  
        // body of instance/non-static method  
    }  
}
```

class is keyword used to define the new class

Variables named varName1 through varNameN are declared inside the class and outside of all methods are called instance variables. Each variable must be assigned a type shown as type1 through

Object Oriented Programming Through Java (Unit- II)

typeN(any primitive data type like int,float,char,..) and may be initialized to a value shown as valueN. if a variable is declared by keyword static then they belong to all objects called as static or class variables (we will cover about static later, in this chapter)

Constructors are used to initialize the instance variables of an object, executed when an object is created. It always has the same name as the class. They will not return any value. A constructor can be defined with or without arguments. We can define more than one constructor for a class.

Methods named mthName1 through mthNameN should be defined with in the class. A static method can be called directly from main/any static method. a non-static method should be called through an object only.

when we want to run a class, a main method should be defined in it with following signature. Java program execution starts from main method. A main method is the starting point for execution by JVM.

```
public static void main(String arguments[])  
{  
}
```

Declaring Objects using new

The *new* operator instantiates a class by dynamically allocating (i.e, allocation at run time) memory for a new object and returning a reference to that memory. This reference is then stored in the variable declared with type class name.

creating objects for class Student - an object for class can be created

using the key word '**new**'

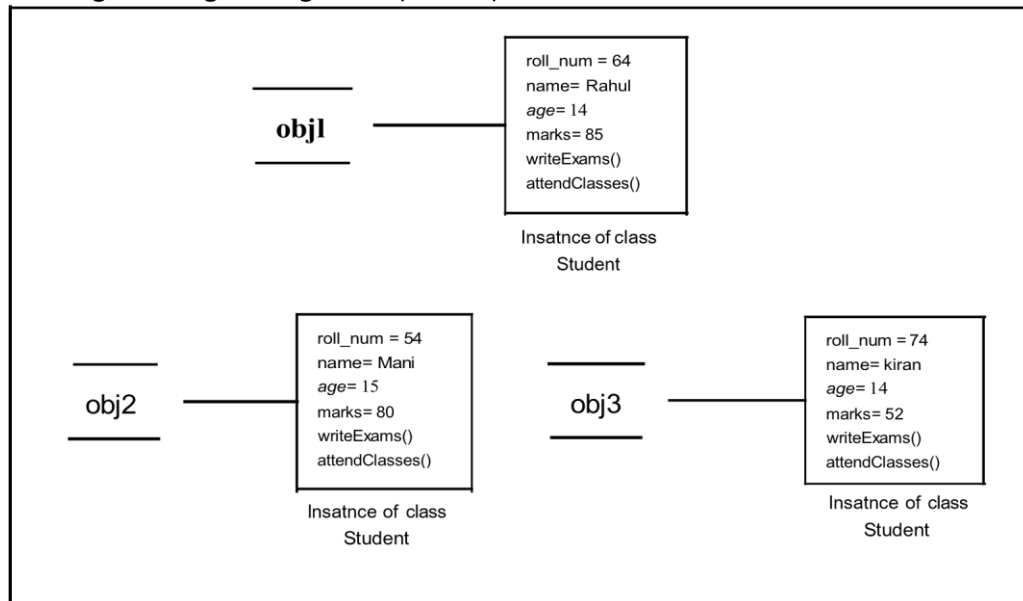
```
Student obj1 = new Student ();
```

```
Student obj2 = new Student ();
```

```
Student obj3 = new Student ();
```

obj1.roll_num = 64;	obj1.roll_num = "Rahul";	obj1.age= 14;	obj1.age = 85;
obj2.roll_num = 54;	obj2.roll_num = "Mani";	obj2.age = 15;	obj2.age = 80;
obj3.roll_num = 74;	obj3.roll_num = "Kiran";	obj3.age = 14;	obj3.age = 52;

Object Oriented Programming Through Java (Unit- II)



As shown in above diagram whenever a new object is created using new, an instance for class will be created, and can be accessed through class reference variables like obj1, obj2, obj3.

Write a program in single java file, which contains two classes Teacher and Student,

Student should have properties roll_num, name and marks. and functions increment, decrement which will increment and decrement marks by 10.

In class Teacher write a main method, in that create two student class objects initialize their properties and later call `increment()` on 1st student object and call `decrement()` on second Student object.

```
class Student
{
    int roll_num;
    String name;
    int marks;
    void
    increment()
    {
        marks = marks + 10;
    }
    void decrement()
    {
        marks = marks - 10;
    }
}
```

Object Oriented Programming Through Java (Unit- II)

```
class Teacher
{
    public static void main(String args[])
    {
        Student s1 = new Student();

        Student s2 = new Student();

        s1.roll_num = 54;
        s1.name = "Rahul";
        s1.marks = 55;

        s2.roll_num = 64;
        s2.name = "Kiran";
        s2.marks = 80;

        System.out.println("initial marks of student "+s1.name+" are "+s1.marks);
        s1.increment();
        System.out.println("after increment marks of student "+s1.name+" are "+s1.marks+"\n");
        System.out.println("initial marks of student "+s2.name+" are "+s2.marks);
        s2.decrement();
        System.out.println("after decrement marks of student "+s2.name+" are "+s2.marks);
    }
}
```

OutPut-

```
initial marks of student Rahul are 55
after increment marks of student Rahul are 65
initial marks of student Kiran are 80
after decrement marks of student Kiran are 70
```

Methods :

A method is a set of statements written within the block to perform certain operation. Methods allow us to **reuse** the code, we can call a method any no of times to repeat the execution of statements under it. A method must be declared within the class. Following is the syntax to define a method.

```
modifier returnType nameOfMethod (Parameter List)
{
    // method body
}
```

Object Oriented Programming Through Java (Unit- II)

}

The components in a method include -

- **modifier** - It defines the access type of the method and it is optional to use.
- **returnType** - Method may return value of any type. if returns nothing type is void.
- **nameOfMethod** - This is name of the method.
- **Parameter List** - The list of parameters, also called as arguments. A method can have any no of arguments. These are optional
- **method body** - The method body defines statements for implementing the functionality.

Let's practice some programs on methods for better understanding.

Write a program with class name MethodExample, this class should contain a method called display, which prints "This is an example for method " and it returns nothing.

```
class MethodExample
{
    void display()
    {
        System.out.println("This is an example for method");
    }
    public static void main(String args[])
    {
        display(); // a non-static method cannot be called without an object
        MethodExample obj= new MethodExample();
        obj.display();
    }
}
```

OutPut:

This is an example for method

Write a program with class name MethodExample, this class should contain a method called display, it takes a String parameter of user name and prints "welcome to methods username" and it returns nothing.

```
class MethodExample
{
    void display(String userName)
    {
        System.out.println("Welcome to methods"+ userName);
    }

    public static void main(String args[])
    {
        MethodExample obj= new MethodExample ();
        obj.display("Rahul");           // string literal is passed directly
    }
}
```

Object Oriented Programming Through Java (Unit- II)

```
String name = "kiran";  
    obj.display(name);           // String literal is stored in a variable and then passed  
    }  
}
```

OutPut:

Welcome to methods Rahul
Welcome to methods kiran

Write a program with class name MyMethod, this class should contain a method called sum, it takes two integer parameters and prints their sum and it returns nothing.

```
class MyMethod  
{  
    void sum(int num1, int num2)  
    {  
        int sum;  
        sum= num1+num2;  
        System.out.println("sum of variables is " + sum);  
    }  
  
    public static void main(String args[])  
    {  
        MyMethod obj = new MyMethod ();  
        obj.sum(2,4);           // integer literals are passed directly  
  
        int i=12,j=4;  
        obj.sum(i,j);           // integer literals are stored in variables and then passed  
    }  
}
```

OutPut:

sum of variables is 6 sum of
variables is 16

Write a program with class name MyMethod, this class should contain a method called sum, it takes two integer parameters and returns their sum, you should print sum value in main method after completing the function call.

```
class MyMethod  
{  
    int sum(int num1, int num2)  
    {  
        int sum;  
        sum= num1+num2;  
        return sum;  
    }  
  
    public static void main(String args[])  
    {  
        MyMethod obj = new MyMethod ();
```

Object Oriented Programming Through Java (Unit- II)

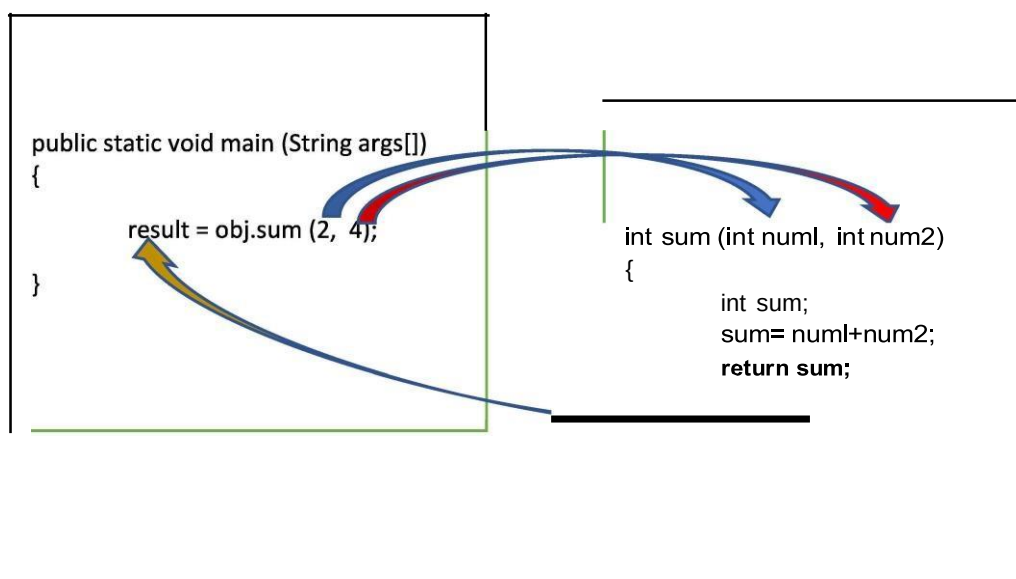
```
int result;  
  
result = obj.sum(2,4);  
System.out.println("received sum value from function "+result);  
  
int i=12, j=4;  
result= obj.sum(i,j);  
System.out.println("received sum value from function "+result);  
}  
}
```

OutPut:

received sum value from function 6
received sum value from function 16

Note:

- * We can return a value or variable with keyword return.
- * It is must to hold the returned value at function call. In our example the sum is returned and received value is captured under the variable result.



Constructors in Java

Constructors are used to initialize the instance variables of class, at the time of creating object.

rules to be followed for defining a constructor

1. Constructor name must be the same as its class name
2. Constructor **should not** be associated with any return type, not even void. A constructor will never return anything.

There are two types of constructors in Java:

1. Default constructor (constructor without any arguments)
2. Parameterized constructor (constructor with arguments)

Default Constructor (constructor without any arguments)

A constructor that has no parameter is known as default constructor. If we don't define a constructor in a class, then compiler creates **default constructor** for that class.

Write a program with class name MyDefaultConstructor, which have a default constructor that prints "My program using default constructor".

```
class MyDefaultConstructor
{
    MyDefaultConstructor ()
    {
        System.out.println("My program using default constructor");
    }

    public static void main(String args[ ])
    {
        MyDefaultConstructor obj = new MyDefaultConstructor();
    }
}
```

OutPut:

My program using default constructor.

Parameterized constructor (constructor with arguments)

A constructor that has parameters is known as parameterized constructor. If we want to initialize variables of the class with your own values, then use a parameterized constructor.

Write a program with class name Box, define a parameterized constructor to initialize instance variables. write a method areRectangle to calculate area of rectangle.

```
class Box {

    int length; int width;

    //Constructor for rectangle when two parameters are passed.
    Box (int l, int w)
    {
        length=l;
        width=w;
    }

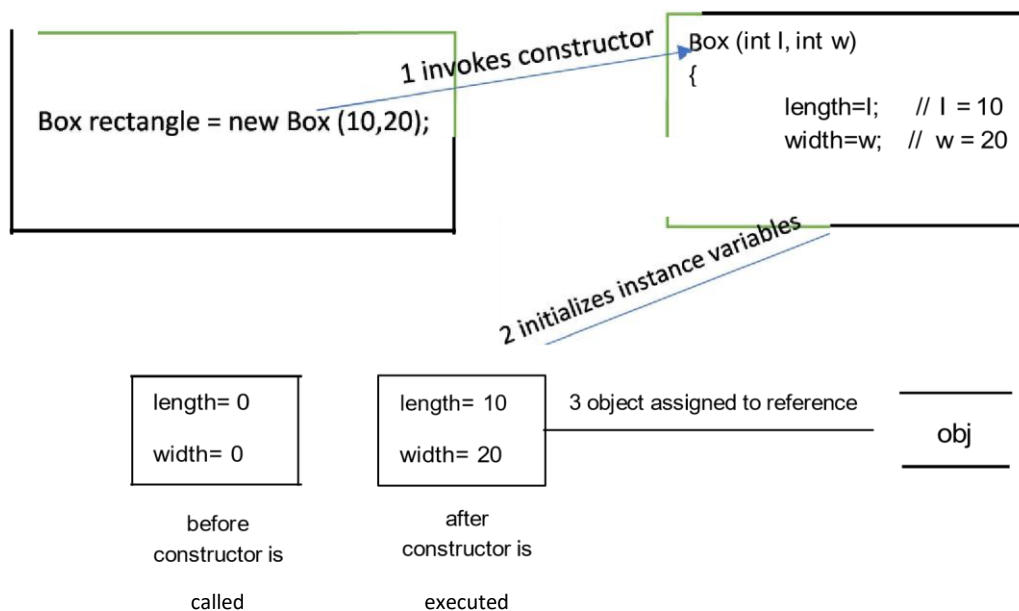
    int areaRectangle()
    {
        return length*width;
    }
}
```

Object Oriented Programming Through Java (Unit- II)

```
}  
  
public static void main(String args[])  
{  
    Box obj= new Box (10,20);  
  
    int area;  
    area = obj.areaRectangle();  
    System.out.println("Area of rectangle is " + area);  
}  
}
```

OutPut:

Area of rectangle is 200



Constructor Overloading

Constructor overloading means defining two or more constructors for same class. The name of all constructors will be same, only number of arguments or type of arguments will be different.

when you pass arguments, the constructor which matches number of arguments and type of arguments will be executed automatically.

Write a program to find area of square and rectangle, with help of constructor overloading.

```
class Box  
{  
    double length;  
    double width;
```

Object Oriented Programming Through Java (Unit- II)

```
//Constructor for rectangle when two parameters are passed.
    Box {double l,double w)      {
        length=l;
        width=w;
    }
//Constructor for square when one value is initialized(area).
    Box (double l) {

        length=l;
    }
    double AreaRectangle() {
        return length*width;
    }
    double AreaSquare()      {
        return length*length*length;
    }
}
//Main class from where program execution begins
class Square_Rectangle_Const_Overloading {
    public static void main (String args[]) {
        Box rectangle = new Box (10,20);
        Box square = new Box (10);
        System.out.println("Area of rectangle is "+ rectangle.AreaRectangle());
        System.out.println("Area of square is "+square.AreaSquare());
    }
}
```

OutPut:

Area of rectangle is 200.0

Area of square is 1000.0

Object Oriented Programming Through Java (Unit- II)

```
class Square_Rectangle_Const_Overloading
{
    public static void main (String args[])
    {
        Box rectangle= new Box (10,20);
        Box square= new Box (10);

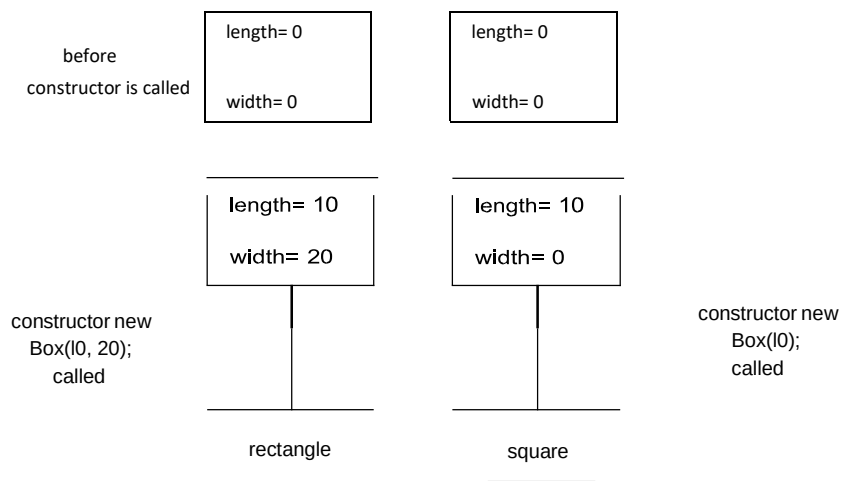
        System.out.println("Area of rectangle is"
                               + rectangle.AreaRectangle())

        System.out.println("Area of square is"
                               +square.AreaSquare()); -----
    }
}

class Box {
    double length;
    double width;
    //Constructor for rectangle of two parameters is called.
    Box (double l,double w) {
        length=l;
        width=w;
    }
    ...//Constructor for square of one value is initialized(area).
    Box (double l) {
        length=l;
    }

    double AreaRectangle() {
        return length*width;
    }

    double AreaSquare() {
        return length*length*length;
    }
}
```



Method Overloading:

Method Overloading (or) static polymorphism allows a class to have more than one method to have the same name, if their argument lists are different.

A method is said to be overloaded, if two methods with same name differ by any of these

Object Oriented Programming Through Java (Unit- II)

1. Number of parameters

example:

```
add(int a, int b)           // function have two arguments
{
    // ...
}
```

```
add(int, int, int)         // function have three arguments
{
    // ...
}
```

2. Data type of parameters

example:

```
add(int a, int b)          // function have two int arguments
{
    // ...
}
```

```
add(int a, float b)        // function have one int argument and one float argument
{
    // ...
}
```

3. Order of parameters

example:

```
add(int a, float b, int c) // function have two arguments
{
    // ...
}
```

```
add(int a, int c, float b) // function have three arguments
{
    // ...
}
```

Write a program to print sum of given numbers, implement it through method overloading.

```
class MethodOverloading
{
    // ...

    void sum (int a, int b)
    {
        int sum;
        sum= a+b;
    }
}
```

Object Oriented Programming Through Java (Unit- II)

```
        System.out.println("method sum declared with two integer arguments");
        System.out.println("sum is "+sum);
    }

    void sum (int a, int b, int c)
    {
        int sum;
        sum= a+b+c;
        System.out.println("this method differs by number of arguments");
        System.out.println("sum is "+sum);
    }

    void sum (int a, float b)
    {
        float sum;
        sum= a+b;
        System.out.println("this method differs by type of arguments");
        System.out.println("sum is "+sum);
    }

    void sum (float b, int a)
    {
        float sum;
        sum= a+b;
        System.out.println("this method differs by order of arguments"); System.out.println("sum
        is "+sum);
    }
    public static void main (String args[])
    {

        MethodOverloading obj = new MethodOverloading();
        obj.sum(2, 4);
        obj.sum(2, 4, 6);
        obj.sum(2, 3.4f);
        obj.sum(3.4f, 2);
    }
}
```

OutPut:

method sum declared with two integer arguments

sum is 6 this method differs by number of arguments

sum is 12 this method differs by type of arguments

sum is 5.4 this method differs by order of arguments

sum is 5.4

Object Oriented Programming Through Java (Unit- II)

Note:

A method will not be overloaded by return type of that method, for example if two methods have same name, same parameters and have different return type, then this is not a valid method overloading.

There will be a compilation error method has been already defined.

Access Control / Access Modifiers

Access control for class members in Java determines who can access or modify the fields, methods, and constructors of a class. This is done using access modifiers: private, default (no modifier), protected, and public.

1. **Private:** The access level of a private modifier is only within the class. It cannot be accessed from outside the class.
2. **Default:** The access level of a default modifier is only within the package. It cannot be accessed from outside the package. If you do not specify any access level, it will be the default.
3. **Protected:** The access level of a protected modifier is within the package and outside the package through child class. If you do not make the child class, it cannot be accessed from outside the package.
4. **Public:** The access level of a public modifier is everywhere. It can be accessed from within the class, outside the class, within the package and outside the package.

Following table describes accessibility of different modifiers in java

	Private	Default		Protected	Public
Same Class	✓	✓		✓	✓
Same package sub-class	X	✓		✓	✓
Same package non-sub-class	X	✓		✓	✓
Different package Sub-class	X	X		✓	✓
Different package Non-sub class	X	X		X	✓

Accessing Private Members of Class:

In Java, private members of a class are not directly accessible outside the class they are declared in. However, you can access those private members using the public Getter and Setter Methods.

Getter Method:

- A getter method allows you to **retrieve** the value of a private field.
- It usually has the name `getFieldName ()`

Example:

```
public String getName()  
{  
    return name;    // Returns the value of the private field 'name'  
}
```

Setter Method:

- A setter method allows you to **modify** the value of a private field.
- It usually has the name `setFieldName (Type fieldValue)`.

```
public void setName(String name)  
{  
    this.name= name; // Sets the value of the private field 'name'  
}
```

Example to access the Person class private variable name in the class Main.

```
class Person {  
    private String name;// Private field  
  
    // Getter method to access the private 'name' field public String  
    getName() {  
        return name;  
    }  
  
    // Setter method to modify the private 'name' field  
    public void setName(String name) {  
        this.name= name;  
    }  
}  
  
public class Main {  
    public static void main(String[] args) { Person  
        person = new Person();
```

Object Oriented Programming Through Java (Unit- II)

```
// Using setter to set the name person.setName("Alice");

// Using getter to retrieve the name
System.out.println("Name: "+ person.getName());

}
```

Garbage Collector is a program in java, which is responsible for de-allocating memory occupied by un referenced objects. Garbage collector will be running periodically to de allocate memory of unused objects.

When a java object is being destroyed, Garbage Collector calls finalize() method on the object to perform clean-up activities(free the resources accessed by that object). Once finalize() method is executed, Garbage Collector will destroy that object.

when an object is said to be ready for garbage collection?

1. object reference variable is made null
2. when object reference variable points to another object

'to be specific when there is not at least one reference to that object'

Write a program implementing finalize () method with any print statement, and make it executed from garbage collector.

```
public class GarbageCollectionExample
{
    public void finalize()
    {
        System.out.println("object is garbage collected");
    }
    public static void main(String args[])
    {
        GarbageCollectionExample s1 = new GarbageCollectionExample();
        s1 = null;
        System.gc();
    }
}
```

OutPut:

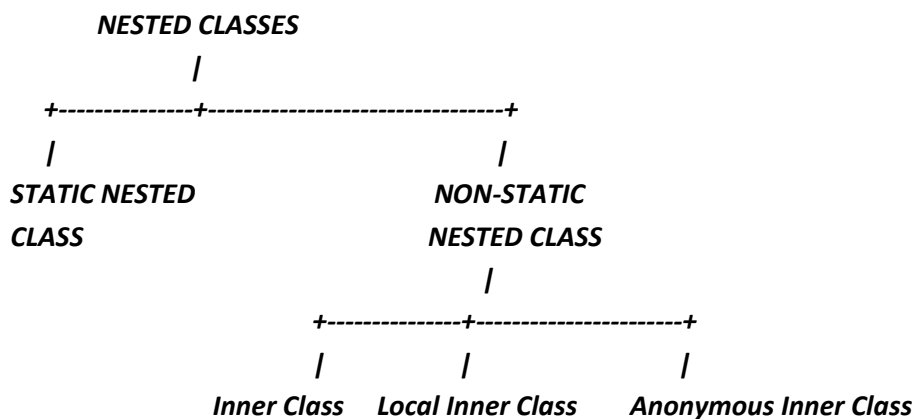
object is garbage collected.

Nested Classes

java allows us to define a class within another class, known as the **nested class**

```
class OuterClass {  
  
class NestedClass {  
  
    }  
}
```

Nested classes are divided into **static** and **non-static**. Nested classes that are declared as static are called **static nested classes**. Non-static nested classes are called **inner classes**.



Note:

Static nested classes do not have access to non-static members of the outer class.

As a member of the outer class, a nested class can be declared **private**, public, protected or can be left default.

Static Nested class:

An Nested class can be declared **static**, which means that you can access it without creating an object of the outer class:

```
class Outer
{
    static class StaticNestedClass
    {
        void innerClassMethod()
        {
```

Object Oriented Programming Through Java (Unit- II)

```
        System.out.println("from static Nested class");
    }
}
public static void main(String args[])
{
    Outer.StaticNestedClass obj2 = new Outer.StaticNestedClass();

    obj2.innerClassMethod();

}
```

Note: inner classes can access attributes and methods of the outer class

Inner Class:

A non-static class created within class and outside all methods

```
class Outer
{
    class InnerClass
    {
        void innerClassMethod()
        {
            System.out.println("from Inner class");
        }
    }

    public static void main(String args[])
    {
        Outer obj1 = new Outer();
        Outer.InnerClass obj2 = obj1.new InnerClass();  obj2.innerClassMethod();
    }
}
```

Note: To create an object for inner class, We must create object for outer class first.

```
OuterClass.InnerClass ref= outerClassObject.new InnerClass();
```

Method Local Inner Class:

An inner class can be defined within a method, just like local variables, the scope of the inner class is within the method. A method-local inner class can be instantiated only within the method.

```
class Outer
{
    void outerClassMethod()
    {
        class Inner
```

Object Oriented Programming Through Java (Unit- II)

```
        {
            void innerClassMethod()
            {
                System.out.println("from method local inner class");
            }
        }
        Inner obj2 = new Inner();
        obj2.innerClassMethod();
    }
    public static void main(String args[])
    {
        Outer obj1 = new Outer();
        obj1.outerClassMethod();
    }
}
```

Anonymous InnerClass

Anonymous classes are declared at the time of instantiation/object creation. An inner class is declared without a class name that's why known to be anonymous inner class. Anonymous classes are mostly used to override methods of interfaces.

```
class Outer
{
    void display()
    {
        System.out.println("from outer class method");
    }
    public static void main(String args[])
    {
        Outer obj1 = new Outer();

        obj1.display();

        Outer obj2 = new Outer()
        {
            void display()
            {
                System.out.println("from anonymous inner class");
            }
        };
        obj2.display();
    }
}
```

Object Oriented Programming Through Java (Unit- II)

Example

```
class AnonymousInnerClassExample
{
    public static void main(String[] args)
    {
        // Create an instance of an anonymous inner class implementing the interface
        Greeting greeting = new Greeting()
        {
            public void sayHello()
            {
                System.out.println("from anonymous inner class");
            }
        };

        // Call the method of the anonymous inner class greeting.sayHello();
    }

    //Interface with a single method interface
    Greeting
    {
        void sayHello();
    }
}
```

this keyword:

The this keyword in Java refers to the current instance of the class where it is used. It is essential for accessing instance variables and methods from within the class. When a class's instance variable and method parameters have the same name, this helps differentiate between them. For example, in a constructor or method, this.variable refers to the instance variable, while variable refers to the parameter passed to the method.

In addition, this can be used to invoke other constructors within the same class. This is known as constructor chaining and is useful for reusing code within the class. By calling another constructor, you can initialize an object with different sets of parameters.

When you need to pass the current object as a parameter, the this keyword in Java can be passed as an argument. This is often done in scenarios where you need to pass the current object to another method or constructor for processing, such as in callback methods, event handling

```
display(this); // current object reference will be passed
```

'this' is a reference, that refers to the current class object.

Object Oriented Programming Through Java (Unit- II)

write a program to initialize instance variables in constructor.

```
class Example
{
    int a;

    float b;

    Example(int j, float k)
    {
        a= j;
        b = k;
    }

    public static void main(String args[ ])
    {
        Example obj = new Example(5,5.4f);
        System.out.println(obj.a);
        System.out.println(obj.b);
    }
}
```

OutPut:

5 5.4

what if variables declared inside constructor j, k are named with same name as instance variables i.e., a, b. then how to initialize instance variables?

```
class Example
{
    int a;
    float b;

    Example(int a, float b)
    {
        a= a;
        b = b;
    }

    public static void main(String args[ ])
    {
        Example obj = new Example(5,5.4f);
        System.out.println(obj.a);
        System.out.println(obj.b);
    }
}
```

Object Oriented Programming Through Java (Unit- II)

OutPut:

0

0.0 though constructor is defined to initialize instance variables, it has no effect because,

inside constructor

a= a;

b = b;

both at lhs and rhs a,b refers to same local variables that are declared inside constructor.

we can refer to instance variables using this operator, that refers to current object class

Example

```
{  
  
    int a;  
  
    float b;  
  
    Example(int a, float b)  
    {  
        this.a= a;  
        this.b = b;  
    }  
  
    public static void main(String args[ ])  
    {  
        Example obj = new Example(5,5.4f);  
        System.out.println(obj.a);  
        System.out.println(obj.b);  
    }  
}
```

OutPut:

5

5.4

this can also be used

1. to invoke current class method.
2. to invoke current class constructor.
3. this can be passed as an argument in the method call.

Invoke current class method using **this**.

```
class ThisMethod
{
    void display()
    {
        System.out.println("from method display");
    }
    void n()
    {
        System.out.println("calling method using this");
        this.display();
    }
}
class ThisExample
{
    public static void main(String args
    {
        ThisMethod a=new ThisMethod();
        a.n();
    }
}
```

OutPut:

calling method using this from
method display

invoke current class constructor

The this() constructor call can be used to invoke another constructor of same class.

Program to invoke no argument constructor from parameterized constructor using **this()**

```
class ThisConstructor
{
    ThisConstructor()
    {
        System.out.println("From Constructor with no arguments.");
    }
    ThisConstructor(int x)
    {
        this();
        System.out.println("controller back to Parameterized constructor");
        System.out.println("value passed to constructor is "+x);
    }
}
```

Object Oriented Programming Through Java (Unit- II)

```
class ThisExample
{
    public static void main(String args[])
    {
        ThisConstructor a= new ThisConstructor(10);
    }
}
```

OutPut:

From Constructor with no arguments.
controller back to Parameterized constructor value
passed to constructor is 10

Program to invoke parameterized constructor from, no argument constructor using this(argument)

```
class ThisConstructor
{
    ThisConstructor()
    {
        this(20);
        System.out.println("controller back to no argument constructor");
    }
    ThisConstructor(int x)
    {
        System.out.println("From Constructor with arguments.");
        System.out.println("value received is "+x);
    }
}
class ThisExample
{
    public static void main(String args[])
    {
        ThisConstructor a = new ThisConstructor();
    }
}
```

OutPut:

From Constructor with arguments.
value received is 20
controller back to no argument constructor

passed as an argument in the method call

The this can also be passed as an argument in function call. It is mainly used in the event handling. Let's see the example:

```
class ThisExample
{
    int a;

    int b;

    ThisExample()
    {
        a= 10;
        b = 20;
    }

    void display(ThisExample obj)
    {
        System.out.println("a = "+obj.a+ " b =" + obj.b); }

    void get()
    {
        display(this);
    }

    public static void main(String[] args)
    {
        ThisExample object = new ThisExample();
        object.get();
    }
}
```

OutPut:

a= 10 b = 20

Assigning one object to another

In Java, assigning one object to another, means copying the reference of an object, from one reference variable to another reference variable of the same class.

```
Student ref1 = new Student(); // object is created, accessed using reference variable ref1
```

```
Student ref2 = ref1;           // the object being accessed by ref1, now can also be  
                               // accessed using the ref2, both are pointing to same single object
```

This means both the reference variables will point to the same object in memory. Any changes made to the object through one reference variable will be reflected when accessing the object through the other reference variable.

Example program :

```
class Person  
{  
    String name; // Field to store the person's name  
    // Method to display the name  
    void display()  
    {  
        System.out.println("Name: " + name);  
    }  
}
```

```
public class Main {  
    public static void main(String[] args) {  
        // Create a Person object and assign a name directly  
        Person person1 = new Person();  
        person1.name = "Ravi"; // Directly setting the name  
  
        // Assign object pointed person1 to person2  
        Person person2 = person1;  
  
        // Display names through both references  
        System.out.println("Before changing name:");  
        person1.display(); // Outputs: Ravi  
        person2.display(); // Outputs: Ravi
```

Object Oriented Programming Through Java (Unit- II)

```
// Change the name using person2

person2.name = "Kiran"; // Directly modifying the name


// Display names again through both references
System.out.println("After changing name:");
person1.display(); // Outputs: Kiran
person2.display(); // Outputs: Kiran
}
}
```

Explanation:

1. Creating Objects:

```
Person person1 = new Person();

Person1.name =" Ravi";
```

creates a new Person object with the name "Ravi". Person1 holds the reference to this object.

2. Assigning References:

- o Person person2 = person1; does not create a new object. Instead, it makes person2 hold the same reference as person1. Now, both person1 and person2 refer to the same Person object in memory.

3. Changing the Object's State:

- o When person2.name = "Kiran"; is executed, it changes the name of the object that both person1 and person2 point to. Since they refer to the same object, the change is reflected when accessing the object through either reference.

4. Output:

- o Before changing the name, both person1 and person2 display "Ravi". o After changing the name, both display "Kiran", demonstrating that both references point to the same object.

Passing Arguments by Value

When we pass a primitive type (like int, char, etc.) to a method, a copy of the value is passed. Changes made to the parameter inside the method do not affect the original value. It is called to be Passing Arguments by Value

```
public class Main
{
    public static void main(String[] args)
    {
        int number= 10;
        System.out.println("Before calling changeValue: " + number); // Outputs: 10
    }
}
```

Object Oriented Programming Through Java (Unit- II)

```
        changeValue(number);

        System.out.println("After calling changeValue: " +number);// Outputs: 10
    }
    static void changeValue(int value)
    {
        value= 20; // Changes the copy of the value
    }
}
```

Before calling changeValue: 10

After calling changeValue: 10

Passing Arguments by Reference (Using Object References)

When you pass an object to a method, the reference (memory address) of the object is passed. This means that the method can modify the object's state (the values of its instance variables) because the reference points to the same object.

```
class Person
{
    String name;
}

public class Main
{
    public static void main(String[] args)
    {
        Person person = new Person();
        person.name= "Ravi";

        System.out.println("Before calling changeName: "+ person.name);
        // Outputs: Ravi changeName(person);

        System.out.println("After calling changeName: " + person.name);
        // Outputs: Kiran
    }

    static void changeName(Person p)
    {
        p.name = "Kiran"; // Modifies the actual object that p points to
    }
}
```

Object Oriented Programming Through Java (Unit- II)

Out Put:

Before calling changeName: Ravi

After calling changeName: Kiran

Class Objects as parameters in methods

'it refers to same concept as passing arguments by reference'.

Passing class objects as parameters in methods in Java means that the method receives a copy of the reference to the object. This allows the method to modify the object's fields, with those changes being reflected in the original object, as both the original reference and the method's parameter reference point to the same object in memory.

```
class Person
{
    String name;

public class Main
{
    public static void main(String[] args)
    {
        Person person = new Person();
        person.name= "Ravi";

        System.out.println("Before calling changeName: "+ person.name);
        // Outputs: Ravi changeName(person);

        System.out.println("After calling changeName: "+ person.name);
        // Outputs: Kiran
    }

    static void changeName(Person p)
    {
        p.name = "Kiran"; // Modifies the actual object that p points to
    }
}
```

Out Put:

Before calling changeName: Ravi

After calling changeName: Kiran

Recursive Methods

A recursive method is a method that calls itself to solve a problem. The method generally has two main components:

1. **Base Case:** The condition under which the method stops calling itself also called to be terminating condition.
2. **Recursive Case:** The part where the method calls itself with modified arguments.

Example:

```
public class Main
{
    public static void main(String[] args) {

        int number= 5;

        int result= factorial(number);

        System.out.println("Factorial of "+number+ " is "+ result);           // Outputs: 120
    }

    // Recursive method to calculate factorial
    static int factorial(int n)
    {
        if (n == 0 || n == 1)
        {
            return 1;
        }

        else
        {
            return n * factorial(n - 1);
        }
    }
}
```

// Base case

// Recursive case: call the method recursively

Nesting of methods:

Nesting of methods refers to calling one method from within the body of another method. This technique allows a method to use the functionality of other methods to perform part of its task. By breaking down complex operations into smaller, specialized methods, nesting promotes code reusability, organization, and clarity, simplifying both the implementation and maintenance of the code.

Example:

```
public class Example
```

```
{  
    public static void main(String[] args)  
    {  
        int length = 5;  
        int width = 3;  
  
        // Call methods that use the common method multiply()  
        int rectangleArea = calculateRectangleArea(length, width);  
        int squareArea = calculateSquareArea(length);  
  
        // Output results  
        System.out.println("Area of rectangle: " + rectangleArea); // Outputs: 15  
        System.out.println("Area of square: " + squareArea);        // Outputs: 25  
    }  
    // Method to calculate the area of a rectangle static int  
    calculateRectangleArea(int length, int width)  
    {  
        // Use the common sub-task method to get the product of length and width return  
        multiply(length, width);  
    }  
  
    // Method to calculate the area of a square static int  
    calculateSquareArea(int side)  
    {
```

Object Oriented Programming Through Java (Unit- II)

```
        // Use the common sub-task method to get the product of side and side return
        multiply(side, side);
    }

    // Common sub-task method to multiply two numbers static int
    multiply(int a, int b)
    {
        return a * b;
    }
}
```

In the above program, nesting is done by calling the multiply method from within the calculateRectangleArea and calculateSquareArea. This also demonstrates how a common task (multiplication) is used within multiple methods.

The calculateRectangleArea method calculates the area of a rectangle by taking two parameters, length and width. It calls the multiply method to find the product of these two dimensions, which gives the area of the rectangle. The computed area is then returned to the caller.

Similarly, the calculateSquareArea method calculates the area of a square. It takes a single parameter, side, which represents the length of one side of the square. The method uses the multiply method to calculate the square of this side length (i.e., side* side) and returns the result, which is the area of the square.

The multiply method is a utility method that performs the multiplication of two numbers. It takes two parameters, a and b, multiplies them, and returns the product. This method is called by both the calculateRectangleArea and calculateSquareArea methods to perform the multiplication needed for their respective calculations. By using a common method for multiplication, the program achieves code reusability and keeps the calculations clean and organized.

Method Overriding:

Method overriding occurs when a subclass (child class) provides a specific implementation of a method that is already defined in its superclass (parent class). The method in the subclass should have the same name, return type, and parameters as the method in the superclass.

1. The overridden method in the subclass must have the same signature (name and parameters) as the method in the superclass.
2. The return type of the overridden method must be the same type as the return type of the method in the superclass.

3. The overridden method can call the method from the superclass using the `super` keyword.

write a program to explain how method overriding takes place between parent and child class

```
class Parent
{
    void display()
    {
        System.out.println("method display of class Parent");
    }
}
public class Child extends Parent
{
    void display()
    {
        System.out.println("method display of class Child");
    }
    public static void main(String args[])
    {
        Child ch = new Child();
        ch.display();// calls child display method
    }
}
```

OUTPUT:

method display of class Child

Note: When we want to access parent, class overridden functions we can do, using `super`.

Recursive Methods in Java

In Java, a method that calls itself is known as a **recursive method** and this process is known as **recursion**.

```
public static void main(String[] args) {
    ... ..
    recurse()
    ... ..
}

static void recurse() {
    ... ..
    recurse()
    ... ..
}
```

Normal Method Call

Recursive Call

Working of Recursion in java

Key Concepts of Recursion

1. **Base Case:** The condition under which the recursion stops. It prevents infinite recursion and eventual stack overflow.
2. **Recursive Case:** The part of the method where the recursion happens. It usually involves a call to the method itself with modified arguments.

Example: Factorial Calculation

The factorial of a non-negative integer n (denoted as $n!$) is the product of all positive integers less than or equal to n . It can be defined recursively as:

- $n! = 1$ if $n \leq 1$
- $n! = n \times (n - 1)!$ otherwise

Base Case: If n is less than or equal to 1, return 1.

Recursive Case: Multiply n by the factorial of $n - 1$.

```
class Factorial {  
  
    public static int factorial( int n ) {  
        if ( n <=1)                // Base case  
  
        return 1;  
        else  
  
        return n * factorial(n-1); // recursive case  
    }  
}
```

```
public static void main(String[] args) {  
    int number = 4, result;  
  
    result = factorial(number);  
    System.out.println(number + " factorial = " + result);  
}  
}
```

Output:

```
4 factorial = 24
```

Advantages:

- Simpler code for problems that are naturally recursive.
- Effective for divide-and-conquer problems.
- Manages state through the call stack.
- Can be more readable and intuitive.

Disadvantages:

- Performance overhead and risk of stack overflow.
- Higher memory usage due to stack frames.
- Harder to debug complex recursive calls.
- Potential inefficiency in cases of redundant computations.

Nesting of Methods in Java

- In Java, method nesting refers to the practice of calling one method from within another method.
- This is a common programming technique used to break down complex operations into simpler, manageable pieces, and to promote code reuse.

Key Concepts of Method Nesting :

1. **Method Call:** When one method calls another method, the calling method is referred to as the **caller**, and the called method is referred to as the **callee**.
2. **Scope:** Methods can call other methods within the same class, or methods in other classes if they are accessible.
3. **Stack Behaviour:** Each method call creates a new stack frame, which allows methods to operate independently in terms of local variables and parameters.

Example:

```
public class MethodNestingExample {  
    public static void main(String[] args) {  
        // Call the main method  
        methodA();  
    }  
}
```

Object Oriented Programming Through Java (Unit- II)

```
public static void methodA() {  
    System.out.println("In methodA");  
  
    // Call methodB from methodA  
  
    methodB();  
}
```

```
public static void methodB() {  
    System.out.println("In methodB");  
  
    // Call methodC from methodB  
    methodC();  
}
```

```
public static void methodC() {  
    System.out.println("In methodC");  
  
}  
}
```

Output:

```
In methodA In methodB In methodC
```

Example with Parameters and Return Values:

```
public class NestedMethodsExample {
```

```
    public static void main(String[] args) {  
        int result = calculateSum(5, 10);  
  
        System.out.println("The result is: " + result);  
    }
```

```
    public static int calculateSum(int a, int b) {  
        int sum = add(a, b); // Calls add method  
  
        int square = square(sum); // Calls square method return square;  
    }
```

```
    public static int add(int x, int y) {  
        return x + y; // Returns the sum of x and y  
    }
```

```
    public static int square(int num) {  
        return num * num; // Returns the square of num  
    }
```

```
}  
}
```

Output:

```
The result is: 225
```

Attribute (Variable) final and static in java

In Java, the final and static keywords can be used to modify class attributes (fields) to achieve specific behaviour.

final Keyword

The final keyword in Java is used to declare constants, immutable variables, and to prevent method overriding or inheritance. When applied to attributes, it indicates that the attribute's value cannot be changed once initialized.

Characteristics of final Attributes:

1. **Immutable:** Once a final attribute is assigned an attribute, it's value cannot be modified. This ensures that the attribute remains constant throughout the lifetime of the program.
2. **Initialization:** A final attribute must be initialized either at the point of declaration or within the constructor of the class.
3. **Usage:** Commonly used for constants or values that should not change after their initial assignment.

static Keyword

The static keyword in Java indicates that a field or method belongs to the class rather than to instances of the class. Static members are shared among all instances of the class and can be accessed without creating an instance.

-----*****-----